LJ INSTITUTE OF COMPUTER APPLICATIONS

LJ University, Ahmedabad – Gujarat

B.Sc. IT (Integrated) – Semester V

Artificial Intelligence: Concepts and Applications (P) – 250021509

AI-Enabled Practical Examination – SET A

Duration: 2 Hours	**Total Marks:** 50 (Practical Performance 40 + Viva 10)
Enrolment No.: _____	**Variant digit d:** _____

Instructions

1. **Variant digit d** = last digit of your enrolment number. Write it in the first cell of your notebook. **Any answer computed for the wrong variant gets zero.**
2. You may use any AI tool. Submit **SetA_<EnrolNo>.ipynb** and **prompts_log.txt** (every prompt, with a timestamp). **If the prompt log is missing, practical marks are capped at 50%.**
3. Every rule in this paper that says "must" overrides your AI tool's default behaviour. Outputs that don't follow the paper's rules get no marks, even if the code runs.
4. Every analysis or comparison must quote **your own printed numbers**. Generic statements get no marks.

Common Scenario: CampusBot – the LJ Campus Helpdesk Agent

Campus map (undirected, unweighted):

Code	Location	Code	Location	Code	Location
G	Main Gate	L	Library	H	Hostel
A	Admin Office	S	Sports Ground	B	Lab Block
P	Parking	C	Canteen	M	Medical Room

Edges: G–A, G–P, A–C, A–L, P–S, C–L, C–H, L–B, S–H, H–M, B–M

Neighbour Expansion Rule (mandatory everywhere): Expand neighbours in **descending order of degree**. Break ties in **reverse alphabetical order** (e.g. S before P).

Expansion count: count a node each time it is taken up for processing (popped from the queue, or entered in a recursive call), including the goal. In BFS, test for the goal **when the node is popped**. In IDDFS, add up the counts across all iterations.

Variant table:

d	0	1	2	3	4	5	6	7	8	9
Start → Goal	G→M	P→B	A→M	S→L	B→P	M→A	G→B	P→M	L→S	C→P

PRACTICAL PERFORMANCE (40 Marks)

Q1. Unit I – Gate Triage Agent (5 Marks)

Write **triage(purpose, hour, is_student, has_appointment, emergency)**. It must return a **node code from the map**, or a message. Rules are checked **in this order, and the first match wins**:

1. emergency → **M**
2. hour ≥ 20 and not a student → "Entry closed – return after 9 AM"
3. purpose = "admission" and has_appointment → **A**
4. purpose = "admission" and no appointment → **A** + "Wait-list token issued"
5. purpose = "food" and 12 ≤ hour < 15 → **C**; purpose = "food" at any other hour → "Canteen closed"
6. student and purpose = "study" → **L**
7. otherwise → **A** + "Visit helpdesk"

(a) Run these 5 cases and add 1 of your own that tests a conflict between two rules: (food, 13, T, F, F), (study, 21, F, F, F), (study, 21, T, F, **T**), (admission, 10, F, F, F), (sports, 16, T, F, F). **[3]**

(b) Admin waiting times (min): 4, 9, 15, 6, 22, 9, 11, 3, 9, 18, 7, 13, plus a **13th value = d + 5**. Using plain loops, compute the mean, population variance, sample variance, and P(wait > 10). Verify with the statistics module, then state the expected number of the next 5 visitors who will wait more than 10 minutes. **[2]**

Q2. Unit II – Campus Route Search (12 Marks)

(a) Build the adjacency list **and** the adjacency matrix programmatically from the edge list. Write neighbours(node) for both, following the Neighbour Expansion Rule. **[1]**

(b) Run BFS from your Start node. Print the visit order level by level (with level numbers), the path to the Goal (using a parent map), and the expansion count. **[3]**

(c) Run bidirectional BFS, alternating **one full level forward, then one full level backward**. Print both frontiers at each step, the meeting node, the full path (start → meet → goal), an edge-continuity check, and the total expansions compared with (b). **[3]**

(d) Implement DLS with a cycle check against the current path. Run it with L = (BFS depth − 1), L = BFS depth, and L = BFS depth + 2, and print the result and expansions for each. Then run IDDFS **reusing your DLS function**, with a trace showing the limit at which it succeeds. **[3]**

(e) Build a comparison table of expansions for recursive DFS (no limit), BFS and IDDFS on your variant. Write a 5-line analysis on completeness, optimality and space, using your numbers. **[2]**

Q3. Unit III – Campus Access Knowledge Base (7 Marks)

Rules (store them as tuples):

Rule	IF	THEN
R1	student ∧ has_id	can_enter_campus
R2	visitor ∧ has_appointment	can_enter_campus
R3	can_enter_campus ∧ library_member	can_enter_library
R4	exam_week	extended_library_hours
R5	extended_library_hours ∧ after_8pm	library_open_late
R6	can_enter_library ∧ library_open_late	allow_route_library
R7	fever ∧ cough	flu_suspected
R8	flu_suspected	visit_medical
R9	visit_medical ∧ after_8pm	call_warden
R10	call_warden ∧ hostel_resident	escort_to_medical
R11	visitor ∧ can_enter_campus ∧ after_8pm	deny_hostel_entry
R12	escort_to_medical	allow_route_hostel_medical

FS1 = {student, has_id, library_member, exam_week, after_8pm}

FS2 = {student, has_id, hostel_resident, fever, cough, after_8pm}

(a) Write a forward-chaining engine. **In each cycle, fire only the lowest-numbered applicable rule whose conclusion is new.** Print a cycle-by-cycle firing log for FS1 and FS2, and stop when no new fact can be added. **[3]**

(b) Add an explanation facility: why(fact) must print the chain of rules and base facts behind any derived fact. Demonstrate it on allow_route_library and escort_to_medical. **[1]**

(c) Re-run with a "fire all applicable rules per cycle" strategy. Compare the number of cycles and the final facts under the two strategies. **[1]**

(d) Build a truth-table evaluator (AND/OR/NOT/IMPLIES). Show that $((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$ is a tautology, and that R9 written as $(v \wedge a) \rightarrow w$ is equivalent to $v \rightarrow (a \rightarrow w)$. **[1]**

(e) In a markdown cell, write R9, R10 and R11 in First-Order Logic with variables and quantifiers (e.g. ∀x Student(x) ∧ ...). **[1]**

Q4. Unit IV – Helpdesk NLP (9 Marks)

FAQ bank:

Doc	FAQ
D1	What are the library timings during exam week?
D2	Where is the medical room and when is the doctor available?
D3	How do I get a duplicate ID card from the admin office?
D4	What are the canteen timings and is food available after 8 pm?
D5	Can visitors enter the hostel after 8 pm?
D6	How do I book the sports ground for a match?

(a) Run an NLTK pipeline on two sentences: "The students were studying in the libraries during the exams." and "Visitors entering hostels after hours are being stopped." Tabulate tokens, tokens after stop-word removal, Porter stems, WordNet lemmas (default), and WordNet lemmas with pos='v'. Pick 2 words where the stem and lemma differ, and say which form suits a chatbot better. **[2]**

(b) Run spaCy POS tagging and NER on "Dr. Rakesh Mehta from LJ University will visit the Ahmedabad campus on Monday." List PERSON/ORG/GPE/DATE entities and point out any wrong or missing label. **[1]**

(c) Build BoW and TF-IDF matrices with stop_words='english', smooth_idf=False, norm=None. Recompute the TF-IDF of "timings" in D4 **in your own code** using idf = ln(N/df) + 1, and match it against sklearn's value. Report D4's top-3 terms and the vocabulary size, and explain why "8" and "after" are missing. **[2]**

(d) Build the chatbot with the default TfidfVectorizer(stop_words='english'), cosine similarity, and threshold **τ = 0.20 + 0.05 × (d mod 4)**. Run the queries below and tabulate the best document, score, and whether the bot answered or fell back. Your bot **must** handle a query with an all-zero vector, and a tie (within 0.01) at the top, by asking a clarifying question. **[3]**

- q1 "library timings in exam week"
- q2 "where can I eat late at night"
- q3 "I lost my ID card"
- q4 "office timings"
- q5 "book a room in hostel"
- q6 "can my parents visit the hostel at night"

(e) Score these with TextBlob and VADER (compound ±0.05 for neutral), then discuss 2 cases where the two tools disagree, or where both are wrong. **[1]**

- "The canteen food is not bad at all."
- "Great, another two-hour queue at the admin office."
- "Library staff were helpful but the wifi was terrible."
- "Hostel rooms are clean."

Q5. Unit V – Integrated CampusBot CLI (7 Marks)

Combine Q1–Q4 into one **campusbot(text, facts)** function, running in an input loop until the user types "exit":

- **Route request** ("from X to Y"): map words to node codes using a synonym dictionary (gate, office/admin, canteen/food court, lab, doctor/dispensary/medical ...). Get the path from your Q2 BFS. Before giving the route, check the KB: going to L requires allow_route_library, and going H→M after 8 PM requires allow_route_hostel_medical. If access is denied, print the explanation from why() or the missing fact.
- **Otherwise:** answer from the Q4 chatbot.

Run these and add one test of your own:

Test	Input	Facts
T1	How do I go from the gate to the library?	FS1
T2	How do I go from the gate to the library?	FS1 without exam_week
T3	I have fever and cough, take me from hostel to the doctor	FS2
T4	where can I eat late at night	{ }

Marks: routing + synonym mapping 2, KB gating with explanation 2, FAQ fallback 1, all test runs shown 1, clean structure/functions 1.

VIVA (10 Marks) – at the Examiner's Desk

Live Modification (4 Marks)

The student draws one sealed card and makes the change in front of the examiner within 8 minutes. AI use is allowed, but the student must explain the result.

Card	Change
M1	Edge C–H is closed for repair. Re-run your BFS and IDDFS and explain the new counts.
M2	Switch to alphabetical tie-breaking. Which of your outputs change, and why?
M3	Add R13: visitor ∧ exam_week → deny_library. Run FS = {visitor, has_appointment, library_member, exam_week, after_8pm}.
M4	Add D7: "The night canteen near the hostel serves food till 11 pm." Re-run q2 and q5 and explain the score changes.
M5	Set τ = 0.40. Which queries now fall back? Is that better or worse?

Technical Questions + Prompt Log (6 Marks)

3 questions from the bank (1.5 marks each), plus 1.5 marks for walking through prompts_log.txt and showing at least 2 places where the AI's first output broke the paper's rules and how the student fixed it.

1. Why did your BFS path differ from an alphabetical-order path?
2. Why does goal-testing on pop or on generation change your expansion count?
3. Why does IDDFS re-expand nodes, yet stay close to BFS in total work?
4. What happens to cosine similarity when the query vector is all zeros?
5. Why does q2 fail even though D4 is the right answer? Give two fixes.
6. When do "lowest-rule" and "fire-all" forward chaining differ?
7. Why did DLS fail at L = depth − 1 but succeed at L = depth?
8. Why does VADER misread "Great, another two-hour queue…"?

Mark Distribution

Unit	Question	Marks	Syllabus practicals covered
I	Q1	5	P-01 – P-03, P-06
II	Q2	12	P-08 – P-18
III	Q3	7	P-19 – P-24
IV	Q4	9	P-25 – P-30
V	Q5	7	P-31 – P-35 (integration)
—	Viva	10	Live modification + questions + prompt log
	Total	**50**	